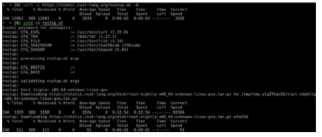

Figure 1: Installation in Linux

```
$ sudo sh rustup.sh
```

To build from source, you need the following prerequisite packages:

- G++ 4.7 or Clang++ 3.x
- Python 2.6 or later (but not 3.x)
- Perl 5.0 or later
- GNU make 3.81 or later
- Curl
- Git

To build from tar, type:

```
$ curl -O https://static.rust-lang.org/dist/rust-nightly.tar.gz
$ tar -xzf rust-nightly.tar.gz
$ cd rust-nightly
```

To build from the repository, type:

```
$ git clone https://github.com/rust-lang/rust.git
$ cd rust
```

After the Rust source code is available, configure it and build using the following commands:

```
$ ./configure
$ sudo make install
```

When the *make* install is complete, several programs will be placed in */usr/local/bin* like *rustc*, the Rust compiler and *rustdoc*, which is the API documentation tool.

Installation in Windows

The precompiled binary installers for Windows are available at the Rust website, from where it can be downloaded. After doing so, run the *.exe* file, which completes the installation of Rust.

You can check whether the installation is complete by typing the following command:

```
$ rustc --version
```

Note: Rust currently needs about 1.5GB of RAM to build without swapping; if it uses swap during the build process, it will take a very long time to build.

Compiling and running the program

Create a file *hello_world.rs* and include the following lines:

```
fn main() {
    println!("Hello, world!");
}
```

The Rust program can be compiled using the following command:

```
$rustc hello_world.rs
```

When the program is compiled successfully, Rust will generate a binary executable automatically, with the name *hello_world* which can be run as usual, as shown below:

```
$ ./hello_world
```

Cargo

Cargo is a tool to manage Rust projects, which has the following functionalities:

- Builds the code
- Downloads the required dependencies for your code
- Builds the dependencies

  To *'cargo-ify'* the project, two steps are to be followed:
- Make a configuration file, *Cargo.toml*
- Put all the source files in the *src* folder

  Given below are the contents of the *Cargo.toml* file:

```
[package]                          // gives the information
of the package to Cargo metadata

name = "hello_world"
version = "0.0.1"
authors = [ "Your name <you@somedomain.com>" ]

[[bin]]                                  // tells that a binary
is to be build not library
name = "hello_world"       // name of the file which is to be
compiled
```

The project can be built and run as follows:

```
$ cargo build
$ ./target/hello_world
```

When the Cargo build is run, you will notice that a file, *cargo.lock*, is created. This file is used by Cargo to keep track of dependencies in the project.

Rust versus C

Even though both the programming languages support manual memory management, they work differently. In C, the memory is allocated and freed by using special functions like